

Quais bibliotecas podem ser utilizadas para trabalhar com arquivos Excel em Python e quais são as diferenças entre elas apresentadas pelas fontes?

Thoughts

Com base nas fontes disponíveis, as bibliotecas utilizadas para trabalhar com arquivos Excel em Python são o Pandas e o OpenPyXL.

As fontes apresentam diferenças claras quanto ao foco, ao nível de abstração e à forma como cada uma interage com as planilhas:

1. Foco e Propósito Principal

Pandas: É uma biblioteca voltada para a análise, exploração, limpeza e manipulação de dados tabulares em geral. O Excel é apenas um dos múltiplos formatos suportados para entrada e saída de dados (junto com CSV, SQL, JSON e Parquet).

OpenPyXL: É uma biblioteca especializada na leitura, alteração e escrita direta de arquivos Excel nos formatos baseados no padrão Office Open XML, tais como `.xlsx` e `.xlsm`.

2. Nível de Abstração e Manipulação dos Dados

Pandas (Alto Nível / Análise em Bloco):

Carrega a planilha inteira para a memória sob a forma de um DataFrame (estrutura bidimensional de linhas e colunas).

Foca na análise e transformação do conjunto de dados como um todo, permitindo aplicar filtragens condicionais, agregações estatísticas, agrupamentos (split-apply-combine via groupby) e limpeza de dados de forma vetorizada, sem a necessidade de percorrer célula por célula manualmente.

OpenPyXL (Baixo Nível / Estrutura da Planilha):

Interage diretamente com a estrutura nativa do arquivo Excel.

Permite abrir e criar pastas de trabalho (Workbooks), acessar abas/planilhas (Worksheets) e manipular valores em células individuais (por exemplo, `ws[ 'A1' ] = 42`).

Permite anexar novas linhas sequencialmente com `ws.append()` e iterar diretamente sobre linhas e colunas com métodos como `ws.iter_rows(values_only=True)`.

3. Integração e Relação Complementar

As fontes mostram que as duas bibliotecas atuam em conjunto: o Pandas utiliza o suporte do OpenPyXL como motor (engine) subjacente para ler arquivos `.xlsx` através da função `pd.read_excel()` e para gravar relatórios finais via `.to_excel()`.

Comparação Prática de Código

Exemplo com Pandas (Análise e Agrupamento)

```
import pandas as pd

# Carrega e manipula a tabela como um todo em memória
dados = pd.read_excel("pagamentos.xlsx") [5]
pendentes = dados[dados["Status"] == "Pendente"] [5]
resumo = pendentes.groupby("Unidade")["Valor"].sum() [11]
resumo.to_excel("relatorio_resumo.xlsx") [11]
```

Exemplo com OpenPyXL (Manipulação Estrutural de Células)

```
from openpyxl import Workbook, load_workbook [2, 10]

# Interação direta com elementos do Excel
wb = load_workbook("relatorio.xlsx") [2]
ws = wb.active [2, 10]
```

```
ws['A1'] = 42 # Atribuição direta a uma célula [10]
ws.append([12-14]) # Adiciona uma nova linha [10]

for row in ws.iter_rows(values_only=True): # Leitura célula por célula [2]
    print(row) [2]

wb.save("amostra.xlsx") [10]
```